

ELECTRICAL CIRCUIT DESIGN FOR A MEDICAL DEVICE

SUMMARY

SHAWN COLIN NUGUID

BENG - TORONTO METROPOLITAN UNIVERSITY

INTERN – POCKET CLINIC INC.

ACKNOWLEDGMENTS

This project was completed in collaboration with Pocket Clinic Inc. where I served as a PCB Design and Verification Intern. I would like to express my sincere gratitude to Dr. Omidvar for their guidance, technical insights, and support throughout the internship.

Please note that due to a signed Non-Disclosure Agreement (NDA), certain proprietary details, including specific device schematics, layouts, design files, and internal testing methodologies, have been intentionally withheld or generalized in this report.

I appreciate the opportunity to contribute to the project and thank Pocket Clinic Inc. for providing a professional environment to develop both technical and collaborative skills.

TABLE OF CONTENTS

1. Introduction	4
2. Additional Notes	4
3. RFID Sensor	5
3.1 – Summary Table	5
3.2 - Polyvinyl Alcohol (PVA)	5
3.3 - Phenanthrene	6
3.4 - ZnO Thin Film	7
3.5 – Final Testing	7
4. PCB Pin Testing	4
4.1 – Matrix and Sequence Drafts	9
4.2 – Matrix Testbench	11
4.3 – Sequence Testbench	12
5. Rubber Degradation Test	14
5.1 - Setup	14
5.2 – Attiny85 Module	15
5.3 - Refining Sequential MOSFET Control	16
5.4 – Feasibility of Using the ESP32 in Place of the Arduino Pro-Mini	17
5.5 – Final Testbench and Troubleshooting	19
6. References	21
7. Appendix	22

1. INTRODUCTION

The main goal of this project is to develop and refine both the hardware and software components of the medical device, ensuring that the device meets all functional and regulatory requirements. This includes tasks such as creating schematics, selecting and sourcing appropriate components, designing the PCB layout, and developing the firmware. By the end of the project, the final product will result in a fully functioning electronic boards in accordance with the devices specifications, complete with detailed schematics and fabricated PCBs. On the software side, there will be final developments, debugs, and optimizing firmware to ensure efficient and accurate device performance, as well as integrate seamless communication between hardware and software.

Key deliverables include:

- (1) selecting reliable and cost-effective components
- (2) creating industry-standard schematic designs
- (3) thoroughly testing and validating the entire system
- (4) documenting all design and testing procedures and ensuring compliance with relevant regulatory standards

Overall, this project provides a comprehensive learning experience in both hardware and software development within the context of medical device engineering.

2. ADDITIONAL NOTES

Due to project constraints, the final formulation and fabrication of the PCB were only minimally addressed. This limitation stemmed from the need to prioritize rigorous testing of the sensor, which presented underlying challenges that required significant time and resources to resolve. Despite this, the project still provided a comprehensive learning experience in hardware and software development within the context of medical device engineering.

Additionally, the following websites were found to be useful in the following research and is recommended to follow up on due to the vast information that can be found:

Matrix and Sequential Code Idea: <https://forum.arduino.cc/t/automated-in-circuit-tester/222223>

Testbench Circuit Troubleshoot: <https://forum.arduino.cc/t/n-channel-mosfet-stays-on/630652>

3. RFID SENSOR

The wireless sensor technologies brought about a revolutionary era of intelligent materials and cost-effective solutions, especially in the context of chipless RFID sensors. The article "*Smart Sensing Materials for Low-Cost Chipless RFID Sensor*" in the IEEE Sensors Journal reports advancements and application of smart materials for quick, broad-band and RFID-enabled monitoring of the surroundings. The authors give a strong argument in favor of the integration of functional materials at a nanostructure level in chipless RFID tags to measure important environmental parameters like temperature, humidity, pH level, gas detection, and mechanical stress in real-time.

These materials, oftentimes referred to as nanostructured functional materials, have a well-known capacity in response to changing environmental conditions. During the project, both Humidity and Temperature were mainly researched to reflect the major concerns of the device.

3.1 – Summary Table

Material	Measure	Testbench
Polyvinyl Alcohol (PVA)	Humidity	Appendix 3A.1 -A.2
Phenanthrene	Temperature	Appendix 3B.1
ZnO Thin Film	Temperature	Appendix 3C.1

Materials NOT Measured but present

Material	Measure	Testbench
Carbon Nanotubes (SWCNTs)	Gas Levels	N/A
Kapton	pH	N/A
PEDOT:PSS	Humidity	N/A

3.2 - Polyvinyl Alcohol (PVA)

Polyvinyl Alcohol (PVA) is a synthetic polymer known for its unique moisture-sensitive properties, making it an excellent candidate for environmental humidity sensing. The molecular structure of PVA is composed of mostly carbon, where each carbon atom is bonded to a hydroxyl (-OH) group. This configuration allows PVA to effectively absorb moisture from ambient air, with the capacity to take up to 25% of its weight in water under high humidity conditions (Amin, Saha, & Karmakar, 2014).

One of the key physical transformations that occur as PVA interacts with water is an increase in its dielectric permittivity. As water molecules permeate the polymer, they enhance its polarization capability, thereby increasing the overall dielectric constant (Amin, Saha, & Karmakar, 2014). Concurrently, the polymer's electrical conductivity also changes, though slightly. The absorbed moisture facilitates limited

ionic conduction, which leads to a marginal reduction in electrical resistance. This conductivity change is inversely proportional to resistance and directly proportional to the amount of humidity present, allowing for predictable and measurable electrical responses under varying environmental conditions.

Another advantageous property of PVA is its relatively high glass transition temperature, around 70°C (Amin, Saha, & Karmakar, 2014). This thermal stability ensures that the polymer maintains consistent behavior under typical ambient conditions, making it suitable for integration into everyday sensing applications.

There are multiple ways to test and verify the effectiveness of PVA as a humidity sensor. Two primary methods are widely used for evaluating its accuracy and responsiveness:

(1) A straightforward yet effective approach to sensing humidity involves monitoring the resistance of a PVA-coated electrode pair. As humidity rises, PVA absorbs moisture and its resistance decreases, causing a measurable change in voltage output when connected to a microcontroller like an Arduino. This setup ensures stable readings and avoids signal noise, unlike direct pin connections. The approach is low-cost, responsive, and well-suited for real-time humidity monitoring in applications such as flexible, chipless RFID systems.

(2) As humidity increases, the capacitance of PVA also rises. This change can be measured by timing how long it takes a capacitor—constructed using PVA-coated electrodes—to charge through a known resistor. To ensure the Arduino can accurately detect these humidity-driven changes in capacitance, a high-value resistor (such as 1 MΩ) should be used to slow the charging process to a measurable duration.

3.3 – Phenanthrene

To evaluate the functionality and accuracy of Phenanthrene as a temperature threshold sensor, an indirect electrical sensing method can be implemented, similar in principle to that used for PVA-based humidity detection. Phenanthrene, a sublimating organic material, undergoes a phase change directly from solid to vapor at approximately 72°C (Amin, Saha, & Karmakar, 2014). This unique property allows it to act as a one-time event detector in temperature-sensitive applications. The sensor configuration involves coating Phenanthrene onto a printed circuit board (PCB) or electrode pair. In this setup, the Phenanthrene forms a temporary conductive path between a digital input pin on an Arduino and ground (GND). Although the Arduino cannot directly detect changes in radio frequency (RF) properties, the sublimation of Phenanthrene can be simulated as a digital signal transition based on the material's altered electrical connectivity. A critical element in this design is the inclusion of a 10kΩ pull-up resistor, which connects the digital input pin to the supply voltage (VCC). This resistor ensures that when the conductive path is broken—i.e., when the Phenanthrene sublimates—the input pin is pulled HIGH, indicating an open circuit. Conversely, while the Phenanthrene is still intact and bridging the connection to ground, the input pin reads LOW, signifying the presence of the conductive path.

In essence, this sensor design allows the system to detect whether the material has reached its sublimation threshold. The digital pin will shift from LOW to HIGH when the material transitions from solid to vapor, thus offering a clear and irreversible indication that a critical temperature has been surpassed.

3.4 – ZnO Thin Film

Zinc Oxide (ZnO), like Phenanthrene, is a promising smart material for temperature-sensitive applications and can be tested using a similar electrical setup. ZnO is a wide bandgap semiconductor whose electrical resistance decreases with rising temperature, making it suitable for indirect temperature detection through voltage monitoring (Amin, Saha, & Karmakar, 2014). As the temperature increases, the resistance decreases. This change alters the voltage at the analog input pin. Specifically, a drop in resistance leads to an increase in the measured voltage at A0 on the Arduino. The Arduino can then interpret this voltage change as a function of temperature, providing a simple yet effective way to monitor thermal variations using ZnO.

Because ZnO offers gradual and reversible resistance changes with temperature, this method enables real-time, continuous temperature monitoring—complementing the irreversible threshold detection provided by Phenanthrene. Both materials can potentially be tested within the same circuit framework, offering dual capabilities for dynamic and event-based thermal sensing.

3.5 – Final Testing

Simulating the behavior of various other smart sensing materials can still be achieved effectively using only the built-in capabilities of the Arduino Uno. By leveraging its analog and digital input functions, a simplified yet functional testing framework can be constructed to mimic how materials like PVA, Phenanthrene, ZnO, and others would respond to environmental factors such as humidity, temperature, pH, and gas exposure.

The core idea is to assign each simulated smart material to a specific Arduino pin and create a unique method to emulate its behavior. Manual inputs—such as jumper wires, potentiometers, or voltage dividers—are used to represent environmental changes. For instance, increasing the voltage across a divider can simulate rising humidity or temperature. These voltages are read through analog or digital input pins and interpreted through basic threshold-based logic in the code.

The following excerpt is based on **Appendix 3D.1**:

In the `setup()` function, each pin is configured according to its expected behavior: digital pins for binary conditions and analog pins for variable responses. Within the `loop()`, the Arduino continuously reads the inputs and compares them to pre-defined thresholds. Conditional logic is then used to simulate material-specific responses. For example:

- If the analog value on the PVA pin exceeds 600, it indicates high humidity.

- If digital pin D2 reads HIGH, it simulates Phenanthrene sublimating due to reaching its threshold temperature.

This approach enables rapid prototyping and demonstration of sensor logic using minimal hardware. It provides a cost-effective platform for education and experimentation, especially when resources are limited.

4. PCB PIN TESTING

Effective pin testing is a critical component of hardware validation, particularly in projects involving custom PCBs or electronic systems with multiple I/O connections. A review of common pin testing strategies revealed four primary approaches, each with unique strengths and limitations depending on the project's size, complexity, and required level of diagnostic precision. These strategies include Pin-to-Pin Testing, Matrix Pattern Testing, Sequence Testing, and Parallel Testing.

(1) Pin-to-Pin Testing - each pin is driven sequentially (either high or low), and all other pins are monitored for expected responses. This technique is ideal for detecting basic faults such as open circuits and shorts. However, the main drawback is its inefficiency when scaling to larger systems, as the number of test operations increases quadratically with the number of pins.

(2) Matrix Pattern Testing- In this method, pins are organized into rows and columns—one set is driven while the other is monitored. This drastically reduces the number of individual drive-and-sense operations, making the process more scalable and time-efficient. While the setup is more complex than Pin-to-Pin Testing, Matrix Testing is highly effective for medium to large systems where testing speed and coverage are priorities, which fits the criteria for the project

(3) Sequence Testing – this method requires a more advanced approach by applying predefined signal sequences to the pins over time. The system then evaluates the output behavior against expected outcomes. This method can detect more subtle issues, such as timing faults or logical inconsistencies that simpler tests may miss. However, it demands a higher level of planning, including the development of validation algorithms and sequencing logic.

(4) Parallel Testing- this is the fastest method, as it drives and monitors multiple pins simultaneously using specialized hardware or software routines. This approach is particularly useful in high-volume production environments where testing speed is critical. The downside is the need for additional hardware and a more complex testing infrastructure. For most development-stage projects, this level of sophistication is unnecessary unless mass production is anticipated.

Based on the evaluation of these strategies, Matrix Pattern Testing and Pin-to-Pin Testing emerge as the most practical and efficient methods for this project's needs. Matrix Pattern Testing significantly reduces test time and is scalable for future development, while Pin-to-Pin Testing provides a reliable foundation for verifying basic connectivity in small-scale or early-stage systems.

4.1 – Matrix and Sequence Drafts

Matrix Testing organizes test pins into logical "rows" and "columns," driving one group (e.g., rows) while monitoring the other (columns). This technique significantly reduces the number of required test cases compared to exhaustive pin-to-pin checks.

A sample Arduino implementation illustrates the concept where each row pin is driven HIGH in sequence, while the corresponding column pins are read. Any unexpected LOW reading indicates an open circuit or short:

```
void matrixTest(int rows[], int columns[], int numRows, int numCols) {
    for (int i = 0; i < numRows; i++) {
        digitalWrite(rows[i], HIGH);
        delay(10); // Signal stabilization
        for (int j = 0; j < numCols; j++) {
            int state = digitalRead(columns[j]);
            if (state != HIGH) {
                Serial.print("Error detected at Row ");
                Serial.print(i);
                Serial.print(", Column ");
                Serial.println(j);
            }
        }
        digitalWrite(rows[i], LOW);
    }
}
```

While Matrix Testing focuses on verifying static connections, Sequence Testing emphasizes the correctness of logical transitions and signal timing. It involves applying a planned series of inputs and verifying that the circuit produces the expected outputs over time.

A simplified Arduino implementation of Sequence Testing uses a structure to define each test step, including the input pin, output pin, expected state, and delay between actions:

```
struct TestStep {
    int inputPin;
```

```
int outputPin;

int inputState;

int expectedOutputState;

int delayMs;

};

void sequenceTest(TestStep steps[], int numSteps) {

    for (int i = 0; i < numSteps; i++) {

        digitalWrite(steps[i].inputPin, steps[i].inputState);

        delay(steps[i].delayMs);

        int actualState = digitalRead(steps[i].outputPin);

        if (actualState != steps[i].expectedOutputState) {

            Serial.print("Sequence error at step ");

            Serial.println(i);

        }

    }

}
```

4.2 – Matrix Testbench

The matrix testing method is designed to operate across a large array of digital pins on an Arduino. The test defines a list of pins and cycles through several basic test patterns, including alternating bits (0xAA and 0x55), all HIGH (0xFF), and all LOW (0x00). For each pattern, the test performs the following sequence:

1. **Pin Configuration**

All listed pins are configured as OUTPUT to ensure full control over their logic states.

2. **Pattern Application**

Each pin is written either HIGH or LOW based on its corresponding bit in the selected pattern. This is achieved by shifting the pattern bitwise and applying the value to each pin in turn.

3. **State Verification**

After setting the pattern, each pin is immediately read back using `digitalRead()` to confirm that its

actual state matches the expected one. If a mismatch occurs, an error is reported on the Serial Monitor, specifying the pin number and the expected versus actual logic level.

Despite its benefits, matrix pattern testing does have inherent limitations. Most notably, it may fail to detect **cross-shorts** between pins—cases where two or more pins are incorrectly connected together but are driven with the same logic level. Since the states of both pins appear correct, the fault goes unnoticed. To address this shortcoming, matrix testing should be supplemented with Pin-to-Pin testing, which explicitly checks for unintended electrical connectivity between every possible pin pair. This added layer ensures complete fault detection, including shorts, opens, and unexpected electrical behaviors.

The code demonstrating the core logic used in this testing approach is located within **Appendix 4A.1**.

4.3 – Sequence Testbench

Automated pin-to-pin testing is a valuable method for verifying the functionality and isolation of digital pins in custom PCBs or microcontroller-based systems. This Arduino-based approach systematically checks each pin for its ability to drive a HIGH output and ensures no unintended electrical shorts exist between pins.

The sequence is as follows:

1. Each pin is tested in sequence by configuring it as an OUTPUT and driving it HIGH.
2. It is then switched to INPUT mode to verify if it can maintain the expected HIGH level—failing this check indicates a self-test failure.
3. Afterward, all other pins are scanned to detect any that falsely read HIGH, which would indicate a short. Pins are then reset to INPUT_PULLUP mode for safety, and the process repeats.

The script tracks and reports the number of passes and failures. It prints error messages in real time:

- If a pin cannot drive HIGH, it reports a self-test failure.
- If a short is detected, it identifies the affected pin pair.

The code demonstrating the core logic used in this testing approach is located within **Appendix 4B.1**.

THIS PAGE LEFT BLANK INTENTIONALLY

5. RUBBER DEGRADATION TEST

A critical component of this project involves the controlled testing of rubber degradation, aimed at understanding the material's behavior under thermal stress. This test is particularly important for applications where rubber components are exposed to heat over time—such as sensors within the device. The degradation test helps determine the material's thermal limits, including its melting point, softening behavior, or irreversible breakdown, which is essential for assessing long-term reliability and safety in real-world conditions.

To simulate how rubber responds to sustained energy input, a programmable test system was developed using an Arduino Uno and multiple N-channel MOSFETs. This system provides an automated way to sequentially apply voltage to test segments, enabling structured and repeatable experimentation. During early testing, LEDs are used in place of rubber heating elements to simulate heat application safely, minimizing the risk of damaging components or creating unsafe testing environments.

5.1 - Setup

The Arduino is programmed to activate each MOSFET in sequence, with each gate connected to pins D2 through D5 via a 220-ohm resistor. For each cycle, a pin is driven HIGH for three seconds, powering the corresponding MOSFET and simulating the application of heat to a rubber segment. After this interval, the pin is turned LOW, followed by a one-second delay before activating the next MOSFET. An LED connected to each output further serves as a visual indicator of the active test cycle.

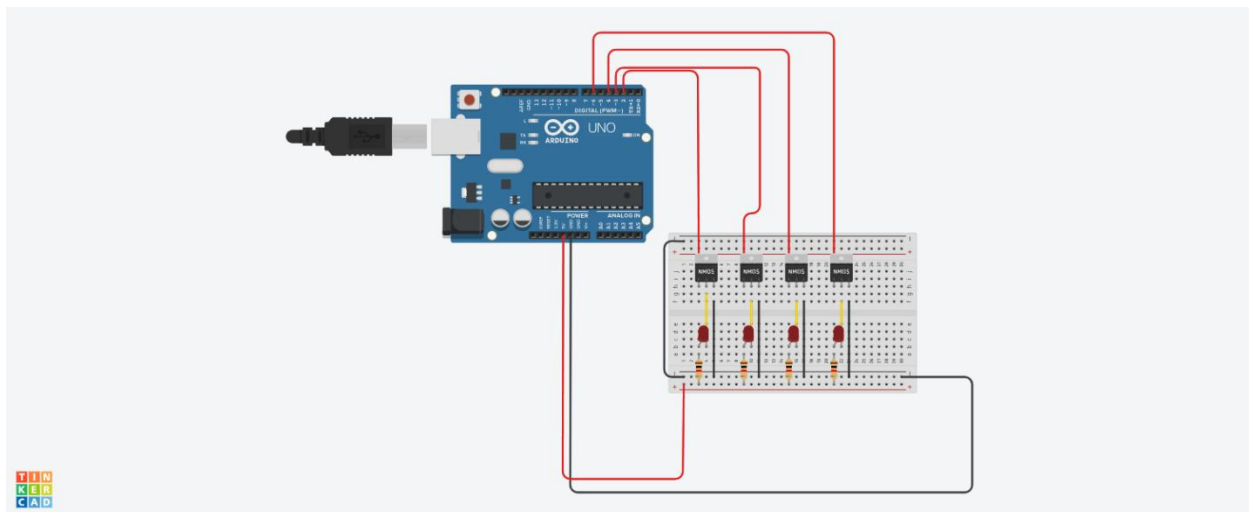


Figure 1 – Sequential Setup of the Rubber Degradation Test

The need for this test arose from a larger effort to evaluate the performance and limits of smart materials used within the devices sensing systems. Rubber, being a common and flexible dielectric or insulating material, is integrated into the wearable device as an embedded sensor mechanical enclosures. However, its physical properties change when subjected to prolonged heat, and in some cases, failure can result in compromised sensor accuracy, structural integrity, or safety. By developing a controlled and programmable test setup, this testbench enables the systematic analysis of thermal wear and failure behavior, supporting material selection and durability assessments.

The testbench code can be found in **Appendix A5.1**.

5.2 – Attiny85 Module

To achieve a more compact and realistic test platform, the original MOSFET control system was redesigned using the ATtiny85, a small but powerful 8-pin microcontroller. Unlike bulkier development boards such as the Arduino Uno, the ATtiny85 provides essential digital I/O functionality in a highly reduced form factor.

Configuration

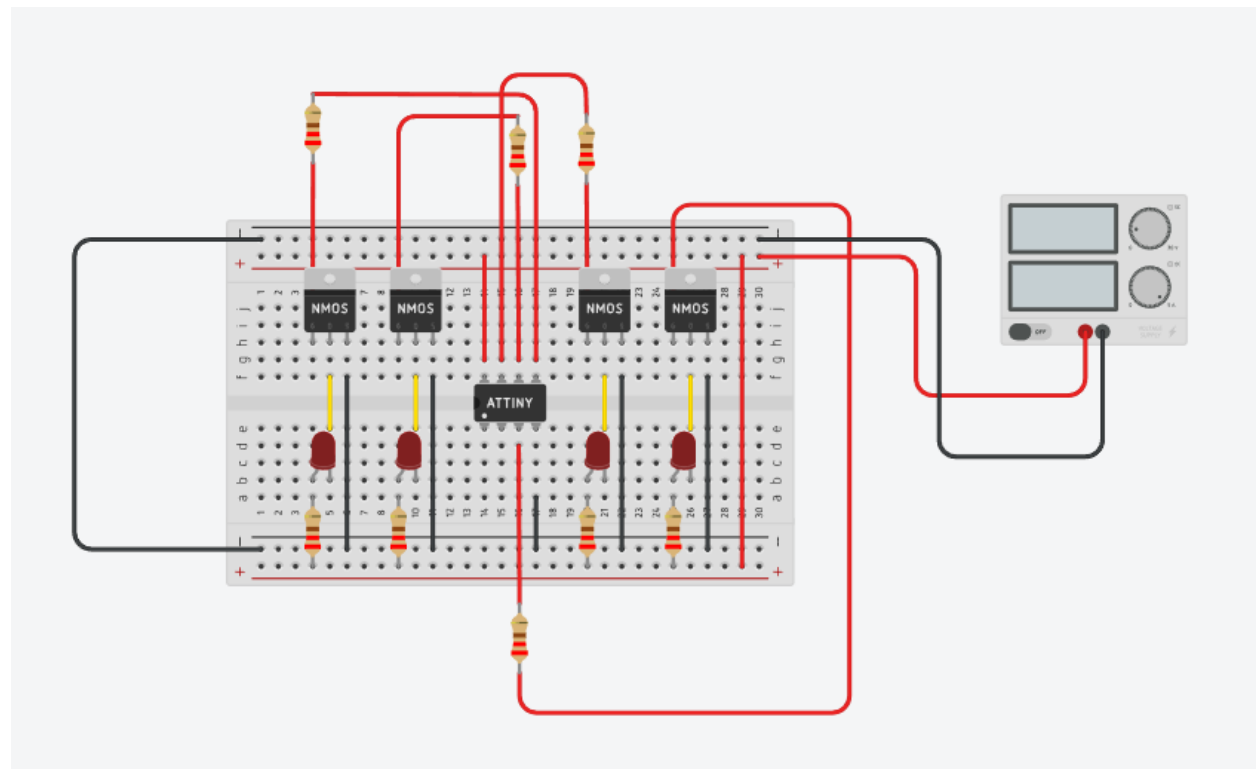


Figure 2 – Rubber Degradation using Attiny85 Module

To ensure long-term reliability and protect the microcontroller's output pins, 220Ω resistors were placed between each ATtiny digital output and the corresponding MOSFET gate. These resistors limit the inrush current during gate charging, a crucial measure for avoiding damage to the microcontroller's limited I/O current capacity. Similarly, identical resistors were also included on the anode side of the indicator LEDs to prevent excessive current draw and potential burnout during activation.

A significant enhancement in this version of the test circuit is the integration of an external power supply to drive the LED loads. Each LED's anode connects to the positive terminal of the power supply via a 220Ω current-limiting resistor. This configuration allows voltage variability, enabling the user to test how the system performs under different supply levels and simulate varying energy conditions safely.

Note: the following expert is in relation to the Testbench Code **Appendix A5.2**.

The control code used to drive the ATtiny85 mimics the previous Arduino version but utilizes pins 0 to 3 to control four MOSFETs. Each pin is set as an output in the setup() function and driven HIGH in sequence for five seconds, with a one-second pause between each activation. This loop repeats continuously to simulate controlled energy delivery or testing intervals.

5.3 - Refining Sequential MOSFET Control: Transition from ATtiny85 to Arduino Nano

Following the successful implementation of the sequential MOSFET activation test using the ATtiny85 microcontroller, the project transitioned back to using the Arduino Uno as reference for enhanced visibility, control, and testing consistency. This phase introduced further refinements in circuit design with consideration also given to the potential use of the Arduino Nano for compact applications as a replacement to the Uno.

Like the ATtiny85 version, this revised design continues to use an external power supply to provide sufficient voltage and current for powering LED loads. However, a critical improvement made in this iteration involves proper ground referencing—a fundamental requirement in MOSFET control. The ground (GND) of the external power supply is explicitly connected to the Arduino GND. This ensures that each MOSFET accurately interprets the gate-to-source voltage, allowing for consistent and reliable switching behavior. Similarly, a 220Ω resistor is placed in series with the anode of each LED, protecting the LEDs from excessive current that could cause immediate burnout during activation. The LED and its corresponding resistor are connected between the positive rail (+V) of the external power supply and the drain of each N-channel MOSFET. The source of each MOSFET is connected to the common ground.

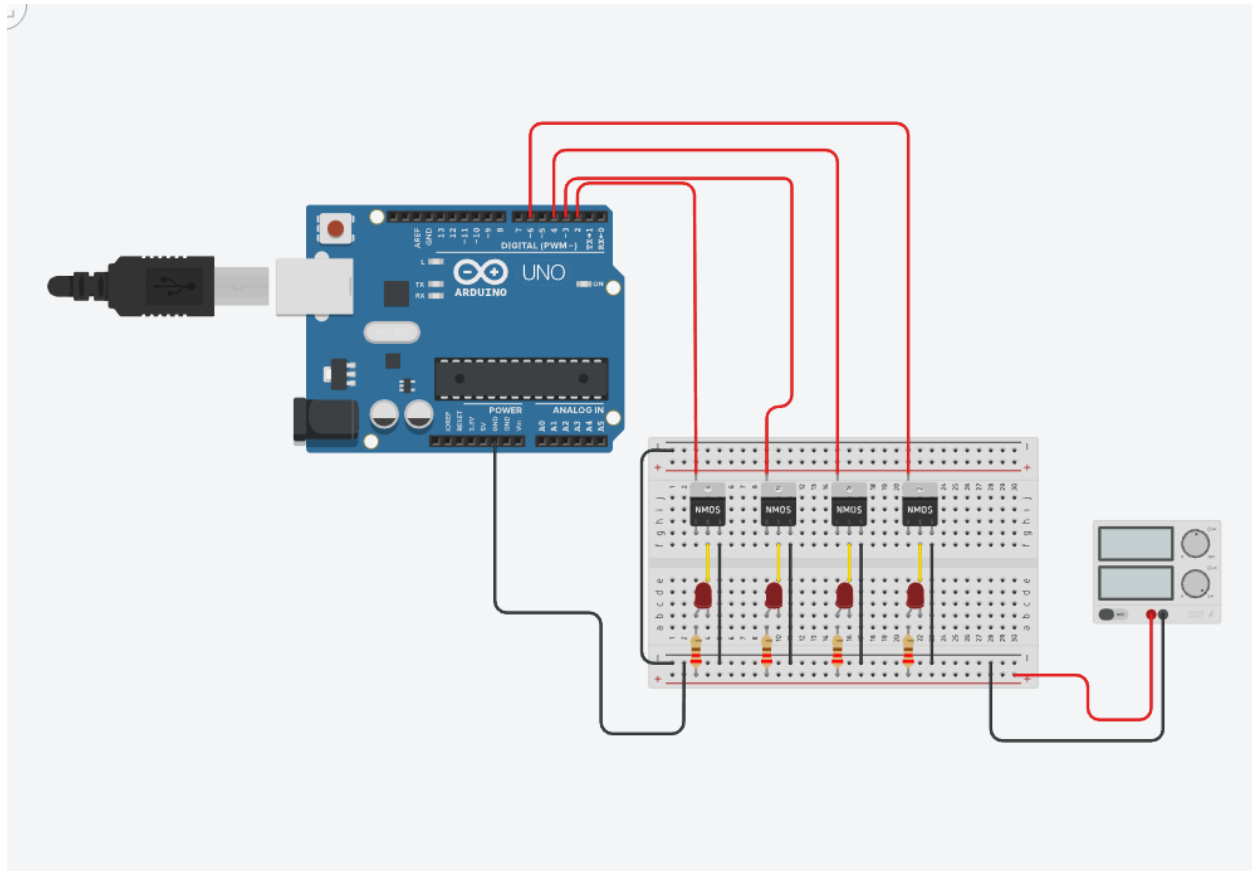


Figure 3 – Improved Rubber Degradation using Uno Module

While the Uno provides a user-friendly platform for testing and debugging, the Arduino Nano offers a more compact and breadboard-friendly alternative without sacrificing performance. Based on the same ATmega328P chip as the Uno, the Nano supports the same codebase and maintains 5V logic levels, making it ideal for direct MOSFET gate control without additional level-shifting hardware.

The logic remains consistent with earlier tests: each MOSFET is turned on in sequence for a defined interval, simulating heating or power delivery cycles. The Arduino sketch used to control the MOSFETs is located within **Appendix A5.3**.

5.4 – Feasibility of Using the ESP32 in Place of the Arduino Pro-Mini

As part of an ongoing effort to improve the functionality and adaptability of the rubber degradation testing system, the feasibility of integrating the ESP32 microcontroller has been carefully evaluated. While the current system operates effectively using the **Arduino Pro-Mini**, this analysis explores whether the ESP32 could serve as a more capable alternative.

a. Project Background and Existing Setup

As a reminder, the rubber degradation testing system involves the sequential activation of heating elements through N-channel MOSFETs, simulating material wear under thermal stress. The Arduino Pro Mini, operating at 5V logic, currently provides reliable control for switching these MOSFETs, specifically the IRL3705 MOSFET (55V, 75A TO-220AB), which requires a minimum of 5V gate voltage to operate in full saturation mode (“IRL3705NPBF | DigiKey Electronics”). This configuration ensures that the heating elements receive stable, high-current power without the risk of underdriven transistors causing thermal inefficiencies or switching failures.

However, as the project evolves, there is growing interest in incorporating features such as real-time monitoring, remote data logging, and wireless control. These advanced capabilities are not supported natively by the Pro-Mini, which lacks onboard communication modules. A reevaluation of the microcontroller choice—specifically, whether the ESP32, with its integrated Wi-Fi, Bluetooth, and expanded I/O functionality, can be adopted as a replacement without compromising switching performance.

b. Benefits and Capabilities of the ESP32

The ESP32 microcontroller is well-regarded for its versatility and built-in connectivity. Its dual-core processor, abundant digital and analog pins, and low-power operation modes make it ideal for modern embedded systems (Espressif). Most notably, its built-in Wi-Fi and Bluetooth allow for wireless data collection, cloud synchronization, and remote system control—features that would significantly enhance the rubber degradation test by enabling continuous logging and monitoring from a distance.

c. Electrical Limitations and Design Challenges

Despite these benefits, the ESP32 outputs at 3.3V logic levels, unlike the Pro Mini, which supports 5V. This difference poses a critical challenge when controlling the IRL3705 MOSFETs, which require a minimum of 5V at the gate to fully switch on. Driving them with only 3.3V from the ESP32 could result in partial conduction, leading to unreliable performance of the degradation test.

While some solutions exist—such as using logic level shifter modules or dedicated gate drivers—they introduce added complexity and cost. Additionally, not all ESP32 breakout boards are tolerant of 5V logic on their GPIO pins, although many, such as the ESP32 DevKit, can accept 5V input at the VIN pin for general power (Espressif).

d. Workarounds and Their Viability

To adapt the ESP32 for this application, two main workarounds are considered:

1. **Logic Level Shifting Modules:** These can safely convert 3.3V GPIO outputs to 5V, allowing full control over the MOSFET gates. However, this approach adds physical bulk and cost, potentially offsetting the benefits of using the ESP32 in a minimal, embedded form factor.

2. **Gate Driver ICs or Transistors:** Using external gate driver circuitry can solve the voltage gap while improving switching efficiency but also increases design complexity and space requirements.

Both approaches are feasible but may be impractical given the project's goals of compactness, simplicity, and low component count.

e. Conclusion and Recommendation

In conclusion, while the ESP32 offers significant functional advantages, especially in terms of wireless communication and system scalability, its use in this rubber degradation testing project introduces non-trivial electrical compatibility challenges. The need to drive 5V gate signals for high-current MOSFETs makes direct use of the ESP32 problematic without additional circuitry. These required modifications could conflict with the project's minimal hardware philosophy. The Arduino Pro Mini, already integrated into the current system, remains the most practical choice due to its 5V logic, ease of integration, and compatibility with the MOSFETs used.

5.5 – Final Testbench and Troubleshooting

As part of a continued effort to develop a compact and automated MOSFET control system, an existing test circuit—originally implemented with the Arduino Uno and Nano—was migrated to an Arduino Pro Mini for space efficiency. The Rubber Degradation Test remained the same throughout each phase:

To sequentially activate a set of N-channel MOSFETs, turning each one on and off at 5-second intervals.

a. Code Functionality Verification

The original control code iterates through four digital pins (D2–D5), each linked to a MOSFET gate. Each pin is driven HIGH for five seconds to activate the corresponding MOSFET, then set LOW for five seconds to deactivate it. This loop ensures sequential switching. Refer to **A5.1 – Rubber Degradation Testbench**.

This code was tested in simulation environments and on other Arduino boards (e.g., Uno), where it operated as expected. These tests confirmed the validity of both logic and wiring.

b. Hardware Investigation and Identified Issues

During real-life, battery-powered operation with the Pro Mini, the MOSFETs remained continuously activated, deviating from the expected timed behavior defined in the program logic. While the code logic

SUMMARY

was sound, several hardware-level factors unique to the Pro Mini contributed to the unexpected continuous activation of the MOSFETs. The following table is a summary of the possible underlying problems and the solutions.

6. REFERENCES

Amin, E. M., Saha, J. K., & Karmakar, N. C. (2014). Smart sensing materials for low-cost chipless RFID sensor. *IEEE Sensors Journal*, 14(7), 2198–2207. <https://doi.org/10.1109/JSEN.2014.2318056>

“IRL3705NPBF | DigiKey Electronics.” *DigiKey Electronics*, 2025, www.digikey.ca/en/products/detail/infineon-technologies/IRL3705NPBF/812272. Accessed 23 May 2025.

Espressif. *ESP32 Series Datasheet Including*. 2024.

7. APPENDIX

3A.1 – PVA Method 1

```
const int sensorPin = A0;

void setup() {
  Serial.begin(9600);
}

void loop() {
  int sensorValue = analogRead(sensorPin);
  Serial.println(sensorValue);
  delay(1000);
}

// Higher reading = lower resistance = higher humidity
```

3A.2 – PVA Method 2

```
void setup() {  
  Serial.begin(9600);  
  pinMode(chargePin, OUTPUT);  
}  
  
void loop() {  
  long startTime, elapsedTime;  
  // Discharge the capacitor  
  pinMode(sensePin, OUTPUT);  
  digitalWrite(sensePin, LOW);  
  delay(10);  
  pinMode(sensePin, INPUT);  
  
  // Start charging  
  digitalWrite(chargePin, HIGH);  
  startTime = micros();  
  while (analogRead(sensePin) < threshold);  
  elapsedTime = micros() - startTime;  
  Serial.print("Charge Time (us): ");  
  Serial.println(elapsedTime);  
  digitalWrite(chargePin, LOW);  
  delay(2000);  
}
```

3B.1 – Phenanthrene Method

```
const int sensePin = 1; //change if applicable

void setup() {
  pinMode(sensePin, INPUT);
  Serial.begin(9600);
}

void loop() {
  int sensorState = digitalRead(sensePin);
  if (sensorState == HIGH) {
    Serial.println("Phenanthrene circuit OPEN (threshold exceeded).");
  } else {
    Serial.println("Phenanthrene circuit CLOSED (below threshold).");
  }
  delay(1000);
}
```

3C.1 – ZnO Method

```
const int sensorPin = A0;
float Vcc = 5.0;
float R1 = 10000.0;

void setup() {
  Serial.begin(9600);
}

void loop() {
  int raw = analogRead(sensorPin);
  float Vout = raw * (Vcc / 1023.0);
  float Rzno = (Vcc * R1 / Vout) - R1;
```



```
Serial.print("Vout: ");  
Serial.print(Vout, 3);  
Serial.print(" V | ZnO Resistance: ");  
Serial.print(Rzno, 0);  
Serial.println(" Ω");  
delay(1000);  
}
```

NOTE: If seen..

- **Voltage increase**
- **ZnO resistance decrease (Rzno)**

This means that the material's electrical and/or optical properties have shifted. In other words, the optical band gap of the material shifts.

3D.1 – Smart Sensor (FINAL) Testbench

```
// --- PIN ASSIGNMENTS ---

#define PVA_PIN A0
#define KAPTON_PIN A1
#define PEDOT_PIN A2
#define ZNO_PIN A3
#define PHENANTHRENE_PIN 2
#define CNT_PIN 3

void setup() {
  Serial.begin(9600);

  pinMode(PHENANTHRENE_PIN, INPUT); // Digital HIGH simulates temp  $\geq 72^{\circ}\text{C}$ 
  pinMode(CNT_PIN, INPUT);        // Digital HIGH simulates gas detected
}

void loop() {
  delay(1000); // Run every second

  Serial.println("--- Smart Material Simulation Test ---");

  // --- PVA Test: Humidity Simulation via Analog Voltage ---
  int pvaReading = analogRead(PVA_PIN);
  Serial.print("PVA Analog Reading: "); Serial.println(pvaReading);
  if (pvaReading > 600) {
    Serial.println(" PVA Response: High humidity, dielectric increased.");
  }

  // --- Kapton Test: Minor Humidity Response ---
  int kaptonReading = analogRead(KAPTON_PIN);
  Serial.print("Kapton Analog Reading: "); Serial.println(kaptonReading);
```

```
if (kaptonReading > 400 && kaptonReading < 550) {  
    Serial.println(" Kapton Response: Minor dielectric change.");  
}  
  
// --- PEDOT:PSS: pH/Conductivity Simulation ---  
int pedotReading = analogRead(PEDOT_PIN);  
Serial.print("PEDOT Analog Reading: "); Serial.println(pedotReading);  
if (pedotReading > 500) {  
    Serial.println(" PEDOT:PSS Response: Conductivity increased (neutral to acidic pH).");  
} else {  
    Serial.println("PEDOT:PSS Response: Low conductivity (alkaline environment).");  
}  
  
// --- ZnO: Temperature Effect on Band Gap ---  
int znoReading = analogRead(ZNO_PIN);  
Serial.print("ZnO Analog Reading: "); Serial.println(znoReading);  
if (znoReading > 500) {  
    float tempSim = map(znoReading, 500, 1023, 25, 100); // Simulated temperature range  
    float bandGap = 3.3 - 0.01 * (tempSim - 25);  
    Serial.print(" ZnO Band Gap Simulation: "); Serial.print(bandGap); Serial.println(" eV");  
}  
  
// --- Phenanthrene: Temp Threshold Simulation ---  
int phenanthreneState = digitalRead(PHENANTHRENE_PIN);  
if (phenanthreneState == HIGH) {  
    Serial.println(" Phenanthrene Response:  $\geq 72^{\circ}\text{C}$  reached, sublimation triggered.");  
}  
  
// --- CNT (SWCNT): Gas Detection Simulation ---
```

```
int cntState = digitalRead(CNT_PIN);  
if (cntState == HIGH) {  
    Serial.println(" Carbon Nanotube Response: Gas detected, conductivity change.");  
}  
Serial.println("-----");  
}
```

4A.1 – Matrix Testbench

```
const int pins[] = {  
    2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13,  
    22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33,  
    34, 35, 36, 37, 38, 39, 40, 41, 42, 43, 44, 45,  
    46, 47, 48, 49, 50, 51, 52, 53  
}; // pin list varies  
  
const int numPins = sizeof(pins) / sizeof(pins[0]);  
  
// --- PATTERNS --- //  
  
const uint8_t patterns[] = {  
    0xAA, // 10101010  
    0x55, // 01010101  
    0xFF, // 11111111  
    0x00 // 00000000  
};  
  
const int numPatterns = sizeof(patterns) / sizeof(patterns[0]);  
  
void setup() {  
    Serial.begin(115200);  
  
    Serial.println("Starting Matrix Pattern Test...");  
  
    // Initialize all pins as OUTPUT  
    for (int i = 0; i < numPins; i++) {  
        pinMode(pins[i], OUTPUT);  
    }  
}  
  
void loop() {  
    for (int p = 0; p < numPatterns; p++) {
```

```
Serial.print("Applying Pattern: 0x");
Serial.println(patterns[p], HEX);
applyPattern(patterns[p]);
delay(10);
verifyPattern(patterns[p]);
}
Serial.println("Testing round complete. Restarting...");
delay(2000);
}
// Apply a pattern to the pins
void applyPattern(uint8_t pattern) {
  for (int i = 0; i < numPins; i++) {

// Set HIGH or LOW based on the corresponding bit (1 = HIGH, 0 = LOW)
    if ((pattern >> (i % 8)) & 0x01) {
      digitalWrite(pins[i], HIGH);
    } else {
      digitalWrite(pins[i], LOW);
    }
  }
}

// Verify if the pins match the expected pattern
void verifyPattern(uint8_t pattern) {
  for (int i = 0; i < numPins; i++) {
    int expected = (pattern >> (i % 8)) & 0x01;
    int actual = digitalRead(pins[i]);
```

```
if (actual != expected) {  
    Serial.print("[ERROR] Pin ");  
    Serial.print(pins[i]);  
    Serial.print(": Expected ");  
    Serial.print(expected);  
    Serial.print(" but got ");  
    Serial.println(actual);  
}  
}  
}
```

4B.1 – Sequence Testbench

```
const int pins[] = {
  2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13,
  22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33,
  34, 35, 36, 37, 38, 39, 40, 41, 42, 43, 44, 45,
  46, 47, 48, 49, 50, 51, 52, 53
}; // pin list varies

void loop() {
  passCount = 0;
  failCount = 0;
  for (int testPin = 0; testPin < numPins; testPin++) {

    // Drive the test pin HIGH
    pinMode(pins[testPin], OUTPUT);
    digitalWrite(pins[testPin], HIGH);
    delay(5);

    bool pinPassed = true;
    // *** Check the test pin itself ***
    pinMode(pins[testPin], INPUT);
    delayMicroseconds(10); // Short stabilization
    int selfState = digitalRead(pins[testPin]);
    if (selfState != HIGH) {
      Serial.print("[FAIL] Pin ");
      Serial.print(pins[testPin]);
      Serial.println(" is not driving HIGH properly!");
    }
  }
}
```



```
pinPassed = false;
}

// Re-drive it high again after testing itself
pinMode(pins[testPin], OUTPUT);
digitalWrite(pins[testPin], HIGH);
delayMicroseconds(10);
// Check all other pins
for (int checkPin = 0; checkPin < numPins; checkPin++) {
    if (testPin == checkPin) continue; // Skip itself
    pinMode(pins[checkPin], INPUT_PULLUP);
    int state = digitalRead(pins[checkPin]);
    if (state == HIGH) {
        // Short detected (another pin went HIGH)
        Serial.print("[FAIL] Pin ");
        Serial.print(pins[testPin]);
        Serial.print(" is shorted to Pin ");
        Serial.println(pins[checkPin]);
        pinPassed = false;
    }
}

// Reset the test pin back to safe state
pinMode(pins[testPin], INPUT_PULLUP);
// Record results to use on Serial Monitor
if (pinPassed) {
    passCount++;
}
```

```
    } else {  
        failCount++;  
    }  
}  
Serial.println("-----");  
Serial.println("Test Completed!");  
Serial.print("PASS: ");  
Serial.println(passCount);  
Serial.print("FAIL: ");  
Serial.println(failCount);  
Serial.println("-----");  
delay(3000);  
}
```

A5.1 – Rubber Degradation Testbench

```
int mosfetPins[] = {2, 3, 4, 5}; // Arduino pins connected to MOSFET gates
```

```
int numMosfets = sizeof(mosfetPins) / sizeof(mosfetPins[0]);
```

```
int onTime = 3000; // 3 seconds ON. This is a variable you can change.
```

```
int offTime = 1000; // 1 second OFF This is variable you can change.
```

```
void setup() {
```

```
  for (int i = 0; i < numMosfets; i++) {
```

```
    pinMode(mosfetPins[i], OUTPUT);
```

```
    digitalWrite(mosfetPins[i], LOW);
```

```
  }
```

```
}
```

```
void loop() {
```

```
  for (int i = 0; i < numMosfets; i++) {
```

```
    digitalWrite(mosfetPins[i], HIGH); // Turn on "burn" for current pin
```

```
    delay(onTime);
```

```
    digitalWrite(mosfetPins[i], LOW); // Turn off
```

```
    delay(offTime);
```

```
  }
```

```
}
```

A5.2 – ATtiny85 Testbench

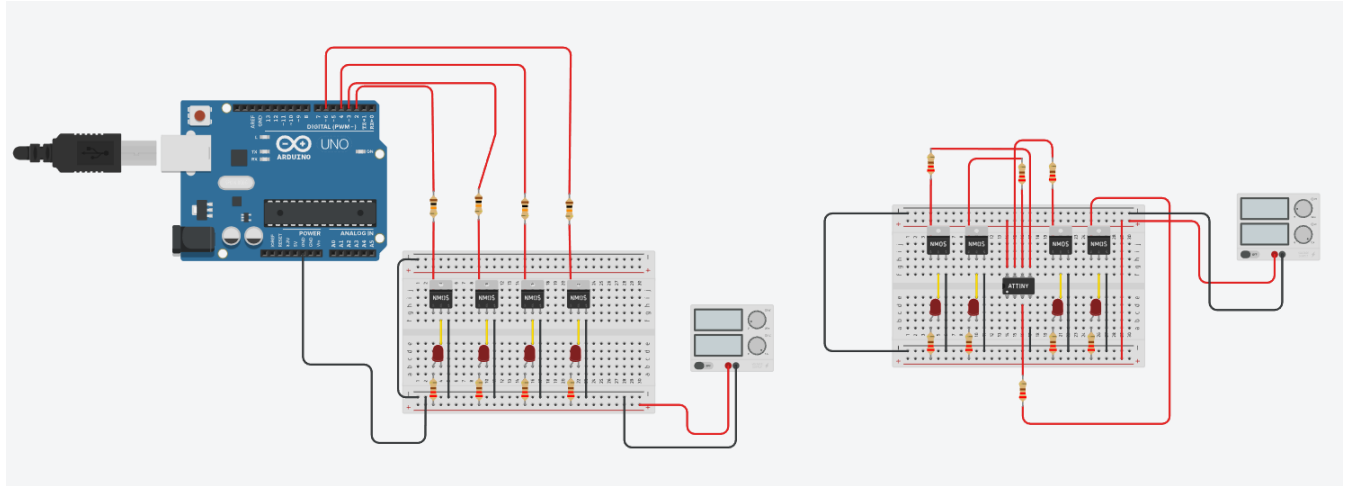
```
int mosfetPins[] = {0, 1, 2, 3}; // Digital pin numbers for P0–P3
int onTime = 5000;
int offTime = 1000;
void setup() {
  for (int i = 0; i < 4; i++) {
    pinMode(mosfetPins[i], OUTPUT);
    digitalWrite(mosfetPins[i], LOW);
  }
}
void loop() {
  for (int i = 0; i < 4; i++) {
    digitalWrite(mosfetPins[i], HIGH);
    delay(onTime);
    digitalWrite(mosfetPins[i], LOW);
    delay(offTime);
  }
}
```

A5.3 – Improved Uno/Nano Testbench

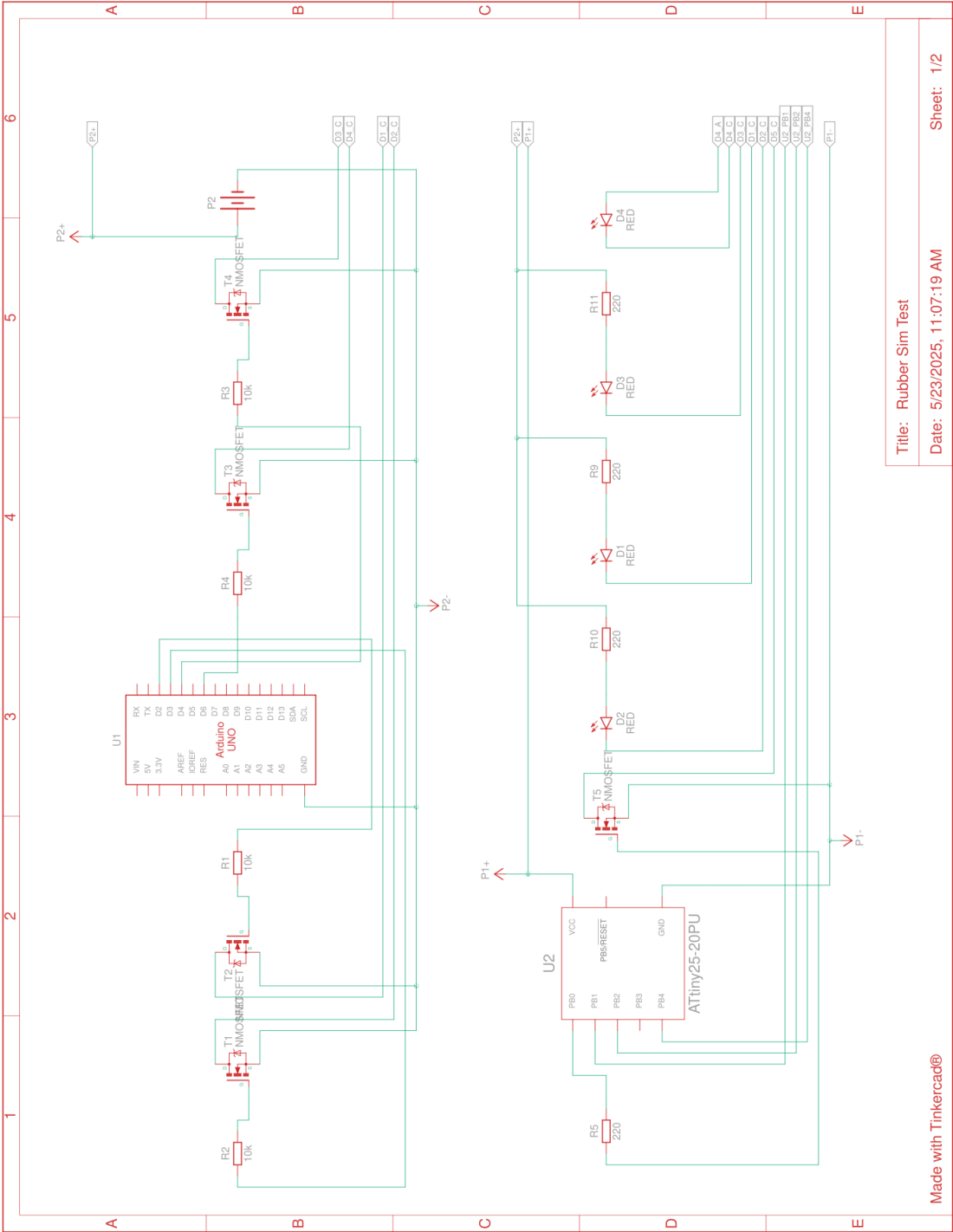
```
int mosfetPins[] = {2, 3, 4, 5}; // Arduino pins connected to MOSFET gates
int numMosfets = sizeof(mosfetPins) / sizeof(mosfetPins[0]);
int onTime = 5000; // 5 seconds ON
int offTime = 5000; // 5 seconds OFF
```

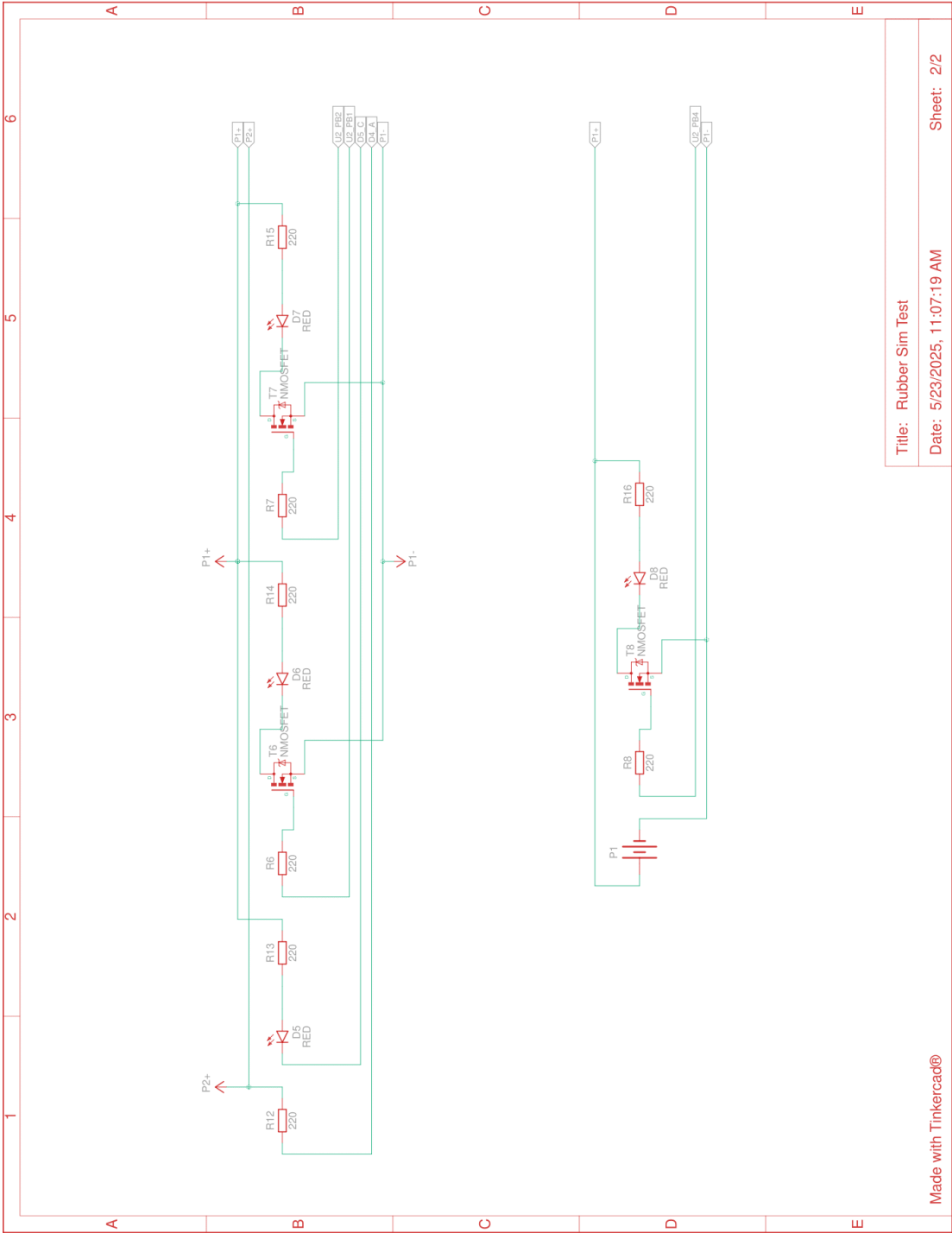
```
void setup() {  
  for (int i = 0; i < numMosfets; i++) {  
    pinMode(mosfetPins[i], OUTPUT);  
    digitalWrite(mosfetPins[i], LOW);  
  }  
}  
  
void loop() {  
  for (int i = 0; i < numMosfets; i++) {  
    digitalWrite(mosfetPins[i], HIGH); // Turn on current MOSFET  
    delay(onTime);  
    digitalWrite(mosfetPins[i], LOW); // Turn off  
    delay(offTime);  
  }  
}
```

A5.5A –Uno and ATtiny85 Circuit Reference Circuit



A5.5B – Uno and ATtiny85 Reference Schematic





THIS PAGE LEFT BLANK INTENTIONALLY